



Warp 2.0: Pulse-Backed Cross-Chain Source Finality

Zach Kelling

Lux Industries

2026-03-03 *Subtitle: An envelope evolution carrying source-chain Horizon finality across chain boundaries via Pulsar Pulses.*

Abstract

We present Lux, a hybrid post-quantum consensus architecture for DAG-native chains. Lux separates fast operational finality from durable post-quantum finality. Validators emit Photons over a Nebula DAG substrate; classical BLS aggregates form Beams for low-latency confirmation, while ML-DSA attestations and Pulsar lattice-threshold Pulses provide post-quantum finality over Nebula roots. Prism binds all certificate lanes to the same transcript, and Quasar reaches Horizon finality when the bound lanes verify.

This paper. Warp 2.0 is the cross-chain envelope that carries source-chain Horizon finality across chain boundaries. Warp 1.x [13] signs cross-chain messages with a BLS aggregate over the source chain’s validator-key registry; that classical-only design gives a destination chain no PQ guarantee about the source’s state. Warp 2.0 adds a Pulsar Pulse over a canonical transcript binding the source chain’s `NebulaRoot`, `KeyEraID`, and `Generation`. The destination verifier runs Prism over the source’s transcript using the source-chain key registry. Beam, ML-DSA, and Pulse all lift their soundness arguments from the consensus side (Quasar Horizon [7, 10]) to the cross-chain side via transcript binding, with one explicit theorem (Warp 2.0 envelope unforgeability, `proofs/quasar/warp-pq-soundness.tex`) covering the cross-chain case. We document the wire format (the canonical ZAP envelope encoding), the v1 $\rightarrow$ v2 migration discipline (forward-only; legacy RLP verifiers reject the ZAP magic), the L2 transitive inheritance, and the constant-time fuzz harness that surfaced one vulnerability (a lattigo `ReadUint64Slice` DoS) which is fixed in production. Warp Private (FHE-encrypted payload, optionally Pulse-signed) is orthogonal and discussed only briefly here.

Contents

1	Introduction	4
1.1	Why Warp 2.0	4
1.2	Forward-only migration	4
1.3	What this paper specifies	5
1.4	What this paper does not do	5
2	Warp Evolution	5
2.1	Warp 1.x: BLS aggregate	5
2.2	Warp 1.5: hybrid (transitional)	6
2.3	Warp 2.0: Pulse-backed cross-chain finality	6
2.4	Warp Private: FHE-encrypted payload	6
2.5	Future PQ proof system	6
2.6	Version table	7
3	Envelope Format	7
3.1	Wire layout	7
3.2	Canonicity rules (the ZAP profile)	8
3.3	The canonical digest D	9
3.4	Lineage is folded into the signed subject	9
3.5	Domain separation	9
3.6	Why <code>HashSuiteID</code> is bound	9
3.7	One parser, no dispatcher	10
3.8	Backward compatibility	10
3.9	Locally-originated messages	10
3.10	No parallel envelope version	10
3.11	Forward-only migration discipline	10

4	Cross-Chain Binding	11
4.1	The three lineage scalars	11
4.2	The GroupKey resolver	11
4.3	Transitive L1 / L2 / L3 inheritance	11
4.4	Cross-chain validator-set evolution	12
4.5	Cross-chain Horizon batching	12
4.6	Replay protection	12
5	Soundness	13
5.1	Warp 2.0 envelope unforgeability	13
5.2	Per-lane reductions	14
5.3	Cross-lane independence	14
5.4	Warp Private orthogonality	14
5.5	Mechanization status	14
5.6	Why we do not assume synchrony for Warp delivery	14
6	Fuzz-Discovered Vulnerability: lattigo ReadUint64Slice DoS	15
6.1	The bug	15
6.2	The fuzz harness that found it	15
6.3	The fix	16
6.4	Why all four layers	17
6.5	Upstream reporting	17
6.6	Test surface	17
6.7	Lessons	17
7	Implementation	18
7.1	Repository layout	18
7.2	The single parser	19
7.3	The PulseVerifier interface	19
7.4	The KernelVerifier flow	20
7.5	Test status	20
7.6	Performance	21
7.7	Cross-implementation parity	21
7.8	Wire-format / lane tests	21
8	Related Work	21
8.1	Chainlink CCIP	21
8.2	Wormhole	22
8.3	IBC	22
8.4	LayerZero	23
8.5	Comparison table	23
8.6	Where Warp 2.0 sits in the cross-chain design space	23
8.7	Status	24

1 Introduction

Lux is not merely adding post-quantum signatures to a chain; it defines a hybrid finality architecture for DAG-native consensus, with protocol-agnostic threshold lifecycle, post-quantum threshold sealing, and cross-chain propagation of Horizon finality.

This paper specifies the cross-chain leg of that thesis. Warp 2.0 evolves the Lux cross-chain messaging protocol (LP-021 [11], github.com/luxfi/warp [15]) to carry source-chain Horizon finality across chain boundaries via a Pulsar Pulse over a canonical envelope transcript. The receiving chain’s verifier runs Prism over the source chain’s transcript, producing a destination-side Horizon-final acceptance whose PQ root of trust resolves to the source’s Pulsar GroupKey, not to a bridge or an oracle.

1.1 Why Warp 2.0

Warp 1.x is shipping (LP-021, LP-075 [11, 13]). It signs cross-chain messages with a BLS aggregate over the source chain’s validator-key registry. The classical-only design has three structural shortcomings:

1. **No post-quantum guarantee.** A quantum adversary that breaks BLS in polynomial time can forge a Warp 1.x envelope from any source chain to any destination, with no fallback PQ check. The destination has no cryptographic basis to discriminate honest from forged Warp 1.x bytes.
2. **No source-finality binding.** A Warp 1.x envelope binds a source chain id, a payload, and a $\geq 2/3$ -weight BLS aggregate. It does not bind *when* on the source the message was finalized: the same source-chain validator set could sign multiple conflicting payloads, and Warp 1.x has no transcript field that anchors the message to a specific source-chain NebulaRoot.
3. **No source-key-era lineage.** A Warp 1.x envelope does not carry the source’s KeyEraID / Generation tuple. A destination that wants to verify against the source’s current key state has to reconcile the envelope’s BLS aggregate against a registry view that may have shifted post-signing.

Warp 2.0 fixes all three. The shape is direct: take the source-chain Horizon certificate (Beam + ML-DSA + Pulse, all bound to the source’s NebulaRoot via the canonical QuasarTranscript); embed it in a cross-chain envelope; bind the envelope’s signed Message to a single canonical digest D , which each lane signs under its own LUX-WARP-ZAP-* domain-separation tag; have the destination verifier run Prism over the source-chain transcript using the source-chain key registry. The result: a cross-chain envelope that, if Prism-accepted, is post-quantum-sound under the same composition theorem (Theorem 1, Section 5) as its consensus cousin.

1.2 Forward-only migration

Warp 1.x and Warp 2.0 share the wire space. The discipline is forward-only:

- A Warp 2.0 envelope begins with the 5-byte ZAP magic "LWZP" || 0x01 followed by the kind byte 0x02. A legacy Warp 1.x RLP decoder fed these bytes rejects: the leading 'L' (0x4c) is below RLP’s list-prefix floor (0xc0–0xff), so it is not a valid RLP list. This is the correct refusal: a legacy verifier cannot validate the ZAP transcript binding, so it must not act on the bytes.
- A Warp 2.0 receiver parses ZAP bytes with one parser, ParseEnvelope (magic, kind, then Message, Beam, and the two PQ lanes). There is no in-band cross-version dispatcher: legacy RLP bytes are rejected at the magic check, never lifted. A locally-originated message with no Pulsar lineage is simply a Message with zero lineage fields and the two PQ lanes carried as empty u32(0) frames — Beam-only semantics without a separate envelope version.

- Senders that must reach legacy v1-only verifiers emit legacy v1 bytes on the legacy channel; the ZAP channel is independent. The ZAP Beam signs `LUX-WARP-ZAP-BEAM-v1 || D` over the full `Message` (lineage included), so it is *not* bit-equal to a legacy v1 BLS aggregate over the old unsigned body — the two channels are cryptographically as well as syntactically distinct.

There is no $v1 \leftrightarrow v2$ negotiation. There is no protocol downgrade. There is no v3 reserved that “maybe re-encrypts the wire.” The only forward step from Warp 2.0 is Warp Private (FHE-encrypted payload, optionally Pulse-signed), which is orthogonal to v2 (Section 2.4).

1.3 What this paper specifies

Section 2 traces the Warp version history ($1.x \rightarrow 1.5$ hybrid $\rightarrow 2.0$ Pulse-backed $\rightarrow$ Warp Private). Section 3 gives the canonical wire format and cites the transcript-binding definition. Section 4 specifies the cross-chain binding fields (`sourceNebulaRoot`, `sourceKeyEraID`, `sourceGeneration`) and the L1/L2/L3 transitive inheritance. Section 5 states the cross-chain soundness theorem and its connection to Quasar Horizon. Section 6 documents the lattigo `ReadUint64Slice` DoS that the fuzz harness surfaced and that is fixed in production. Section 7 cites the running Go canonical (`warp/codec.go`, `warp/zap.go`, `warp/envelope.go`, `warp/pulsar/pulsar.go`) and the wire-format / lane-verification tests. Section 8 compares to CCIP, Wormhole, IBC, and LayerZero, with emphasis on the Pulse-backed PQ source finality angle.

1.4 What this paper does not do

We do not specify the source-chain Horizon construction (that is [7]). We do not specify Pulsar (that is LP-073 [12, 6]) or LSS (that is [18, 14]). Where the underlying threshold mathematics is needed, we cite the canonical proof-bucket files (`proofs/quasar/warp-pq-soundness.tex`, `proofs/pulsar/unforgeability.tex`, `proofs/definitions/transcript-binding.tex`). We do not duplicate proof bodies; we cite them.

2 Warp Evolution

Lux Warp evolves on the same lane structure as Quasar finality. This section traces the version history.

2.1 Warp 1.x: BLS aggregate

LP-021 / LP-075 [11, 13]. Source-chain validators emit a `BitSetSignature` (signer bitmap plus aggregated BLS signature, 96 bytes total) over an `UnsignedMessage` that binds the source `networkID`, `sourceChainID`, and a `payload`. The destination chain verifies the aggregate against the source’s registered validator set (BLS public keys, weights). Warp 1.x is shipping; the corresponding code lives at `github.com/luxfi/warp/message.go`, `signature.go`, and `validator.go` [15].

What 1.x gives. Classical fast cross-chain. Sub-millisecond destination-side BLS aggregate verify (Section 7). Operationally simple: one signing primitive, one verifier.

What 1.x does not give. (1) Post-quantum guarantee on the cross-chain envelope: a quantum break of BLS forges any cross-chain message. (2) Source-finality binding: no transcript field anchors the envelope to a specific source-chain bundle root. (3) Source-key-era lineage: the validator set the destination resolves at verify time may differ from the validator set the source signed with.

2.2 Warp 1.5: hybrid (transitional)

Warp 1.5 was a transitional design that added an optional ML-DSA cert set alongside the BLS aggregate. It bound to the source chain id and payload but not to a Nebula root or a key-era lineage. In production it added per-validator PQ accountability without the compact PQ threshold seal.

Status. Superseded by Warp 2.0. Code never shipped to mainnet; the design persisted only long enough to validate that the Pulse field could compose with the existing `Message` shape without breaking the Warp 1.x BLS path. The Mar-3 freeze ships Warp 2.0; Warp 1.5 is documented here for historical completeness.

2.3 Warp 2.0: Pulse-backed cross-chain finality

The current target, shipped over the ZAP wire (Section 3). A Warp 2.0 envelope carries:

- a single signed `Message` that folds the former unsigned body (`networkID`, `sourceChainID`, `payload`) together with the four source-chain lineage fields (`sourceNebulaRoot`, `sourceKeyEraID`, `sourceGeneration`, `HashSuiteID`) — so the BLS Beam now signs over the *whole* subject and authenticates the lineage it previously left unsigned;
- the Beam (BLS aggregate) over “LUX-WARP-ZAP-BEAM-v1” $\parallel D$, where D is the canonical digest of `Message`;
- one Pulsar Pulse over “LUX-WARP-ZAP-PULSE-v1” $\parallel D$;
- optionally an ML-DSA cert set (per-validator FIPS-204 attestations) over “LUX-WARP-ZAP-MLDSA-v1” $\parallel D$;

The destination verifier (`warp/pulsar/KernelVerifier` [15]) resolves (`sourceChainID`, `sourceKeyEraID`, `sourceGeneration`) to the source’s `GroupKey $\eta$` via the destination chain’s source-chain key registry; runs `Pulsar.Verify` on the Pulse; runs the BLS aggregate verify on the Beam; runs the ML-DSA cert-set verify if the lane is present; returns Prism-accepted iff all lanes verify against the same digest D .

2.4 Warp Private: FHE-encrypted payload

Z-Chain FHE ciphertext envelope. The Pulsar pulse signs the ciphertext digest; the destination verifies under unchanged `GroupKey $\eta$` without seeing plaintext. Warp Private is *production-research*, orthogonal to Warp 2.0; the two compose: a Warp Private envelope MAY carry a Pulse, in which case the v2.0 soundness theorem (Theorem 1) applies unchanged to the encrypted-payload binding. See Remark 2 of `proofs/quasar/warp-pq-soundness.tex`.

2.5 Future PQ proof system

The current Z-Chain rollup uses Groth16 for ML-DSA cert-set compression; a future Warp Private PQ replaces the pairing-based SNARK with P3Q [3] or a lattice-based post-quantum proof system. Until that lands, do not classify the Groth16-wrapped lane as post-quantum (Remark 1, Section 5).

2.6 Version table

Version	Signing lane(s)	Status	Description
Warp 1.x	Beam (BLS aggregate)	Shipping	Classical cross-chain. No PQ guarantee.
Warp 1.5	Beam + optional ML-DSA	Superseded	Hybrid transitional; never deployed.
Warp 2.0	Beam + ML-DSA + Pulse	Shipping (Mar-3 freeze)	Pulse-backed source finality. PQ-sound under the conjunctive Prism predicate.
Warp Private	Pulse over FHE ciphertext digest	Production-research	Confidentiality + PQ source finality, orthogonal composition.
Warp Private PQ	Pulse + P3Q [3] / lattice ZK wrapper	Future	Replaces Groth16 wrapper for true PQ succinctness.

The Mar-3 freeze ships Warp 2.0 in production. Warp 1.x continues to ship for backwards compatibility; the v1 channel and the v2 channel coexist without negotiation, per the forward-only migration discipline (Section 1).

3 Envelope Format

This section gives the canonical wire format for the Warp 2.0 envelope as it ships in `github.com/luxfi/warp v1.22.0`. The wire is *ZAP* — the Lux Warp Zap Protocol — a total-order canonical TLV encoding, *not* RLP. The single source of truth is the canonical Go (`warp/codec.go`, `warp/zap.go`, `warp/envelope.go`, `warp/message.go`, `warp/signature.go` [15]). Where this paper and the Go source disagree, the Go source is the tiebreaker.

3.1 Wire layout

A serialized envelope is a flat byte stream with no recursion and no version dispatch:

```

magic ("LWZP" || 0x01)           // 5-byte wire magic (rule 9)
|| kind (0x02)                   // kindEnvelope discriminator
|| Message                       // the signed subject (ZAP c14n; see below)
|| Beam                         // BLS aggregate lane (BitSetSignature)
|| PulseSig                     // u32-len || bytes (Pulsar Pulse; u32(0) if absent)
|| MLDSACertSet                 // u32-len || bytes (ML-DSA cert set; u32(0) if absent)

```

The four top-level components — `Message`, `Beam`, `PulseSig`, `MLDSACertSet` — are the fields of the Go `Envelope` struct (`envelope.go`). `Envelope.Bytes()` produces exactly the stream above; `ParseEnvelope` is its only inverse.

The signed subject. `Message` is the single signed subject. It folds the former unsigned body (`NetworkID`, `SourceChainID`, `Payload`) together with the source-chain Pulsar lineage (`SourceNebulaRoot`, `SourceKeyEraID`, `SourceGeneration`, `HashSuiteID`) that earlier designs carried as separate, Beam-unauthenticated envelope fields. Its canonical encoding (`Message.marshalZAP()`, identical to `Message.Bytes()`) is the digest preimage:

```

zapKindMessage    u8           // 0x01 (distinct from the 0x02 envelope kind)
NetworkID         u32          // big-endian
SourceChainID     [32]byte     // raw, no length prefix

```

```

SourceNebulaRoot    [32]byte // raw, no length prefix
SourceKeyEraID      u64      // big-endian
SourceGeneration    u64      // big-endian
HashSuiteID         u32-len || utf8 // e.g. "Pulsar-SHA3"
Payload             u32-len || bytes

```

The Beam lane. Beam is a `BitSetSignature` (`signature.go`): the signer bitset plus the 96-byte BLS aggregate.

```

Signers      u32-len || trim-canonical bitset bytes // rule 6
Signature    [96]byte                               // raw BLS aggregate

```

3.2 Canonicity rules (the ZAP profile)

ZAP is total-order canonical: a given struct value has exactly one byte encoding, and decode rejects any stream that is not in canonical form. That property is what makes the encoding safe as a signing domain — every byte is committed and there is no malleability lane. The rules enforced by `zap.go`:

1. Integers are fixed-width big-endian (`u8/u16/u32/u64`). No varints.
2. Every variable-length field is framed with a `u32` big-endian length prefix followed by exactly that many bytes (`appendVar / varbytes`).
3. Fixed-width arrays (`[32]`, `[96]`) are written raw, with no length prefix.
4. Every field is always present. An absent optional lane is the `u32(0)` empty frame, never an omitted field.
5. Booleans are exactly `0x00` or `0x01`; any other byte is rejected.
6. The **Signers** bitset is trim-canonical: no trailing zero byte. A bitset whose final byte is zero is non-canonical (two encodings for one set) — trimmed on encode, rejected on decode.
7. Decode rejects trailing bytes: the cursor **MUST** land exactly on the end of the buffer.
8. No pointers, padding, flags, or maps appear in the format.
9. The stream begins with the 5-byte magic `"LWZP" || 0x01`.

The big-endian length-prefix discipline is lifted from the proven Horizon certificate codec (`warp/pulsar/pulsar.go appendBELenPrefixed / readBELenPrefixed`); in the root package it is the single envelope codec.

Magic and kind bytes. The wire magic is `"LWZP" || 0x01` — the four ASCII bytes L W Z P (Lux Warp Zap Protocol) followed by the format-version byte `0x01`. Immediately after the magic is a one-byte kind discriminator: `kindEnvelope = 0x02`. The `Message c14n` stream independently leads with `zapKindMessage = 0x01`. The two discriminators are deliberately distinct so a standalone `Message` can never be parsed as a full envelope and vice-versa. These three constants — the magic, `0x01`, and `0x02` — are frozen; they are never renumbered.

Field count is fixed. All four top-level components are unconditionally present. An absent PQ lane is the empty `u32(0)` frame, not an omitted field. Decoders only inspect lengths; the stream cannot ambiguously truncate.

Maximum size. `MaxMessageSize = 256 KiB` bounds a `Message`'s canonical encoding; `MaxEnvelopeSize = 4 * MaxMessageSize` bounds the full envelope, leaving room for the Pulse (~ 33 KB) and the ML-DSA cert set alongside the message and Beam (`codec.go`). Both are hard ceilings checked at the decode boundary.

3.3 The canonical digest D

Every lane signs the same digest D under its own domain-separation tag. D is the message ID, the cross-chain replay key, and the on-chain `messageHash` all at once:

$$D = \text{keccak256}(\text{messageDST} \parallel \text{c14n}(\text{Message})), \quad \text{messageDST} = \text{"LUX-WARP-ZAP-CORE-v1"},$$

where `c14n(Message)` is the canonical encoding of the previous subsection (`Message.Bytes()`) and `keccak256` is Ethereum’s `keccak256` (golang.org/x/crypto/sha3.NewLegacyKeccak256, Keccak padding `0x01`) — *not* NIST SHA3-256 (padding `0x06`) and *not* `crypto/sha256`. The on-chain `keccak256` opcode computes exactly this, so D matches byte-for-byte between Go and Solidity. D is always recomputed from the struct (`Message.ID()`), never sliced out of the wire.

Each lane signs D prefixed by its own frozen tag:

- **Beam (BLS)**. signs `BeamSigningBytes(D) = "LUX-WARP-ZAP-BEAM-v1" || D`.
- **Pulse (Pulsar)**. signs `PulseSigningBytes(D) = "LUX-WARP-ZAP-PULSE-v1" || D`.
- **ML-DSA cert set**. signs `MLDSASigningBytes(D) = "LUX-WARP-ZAP-MLDSA-v1" || D`.

These four tags (`codec.go`) are frozen and never altered. Because `c14n(Message)` commits `NetworkID`, `SourceChainID`, `SourceNebulaRoot`, `SourceKeyEraID`, `SourceGeneration`, `HashSuiteID`, and `Payload`, signing over D binds every one of those fields in *every* lane — the Beam included.

3.4 Lineage is folded into the signed subject

This is the structural fix over earlier Warp designs. Previously the Beam signed only the unsigned body and the PQ-lineage fields lived on the envelope, unauthenticated by the BLS aggregate. Folding the lineage into `Message` means D commits it and every lane authenticates it: a Beam over `"LUX-WARP-ZAP-BEAM-v1" || D` now attests to the source chain’s `NebulaRoot`, key-era, generation, and hash suite, not merely to the payload. There is no Beam-unauthenticated field anywhere in the envelope.

3.5 Domain separation

A signature in one lane can never be replayed into another, and a Warp envelope signature can never be confused with a consensus-side signature. Two layers of separation:

- **Intra-Warp**. `LUX-WARP-ZAP-BEAM-v1`, `LUX-WARP-ZAP-PULSE-v1`, and `LUX-WARP-ZAP-MLDSA-v1` are distinct, so even though all three lanes sign the same D , a Beam cannot be lifted into the Pulse lane (and vice-versa). The classical and lattice objects are already non-interchangeable; the distinct tags close the door regardless.
- **Cross-protocol**. The Warp tags differ from the consensus tags: `QUASAR-PULSAR-BUNDLE-v1` signs Nebula bundle roots in consensus, and `QUASAR-PULSAR-ACTIVATE-v1` signs Pulsar key-era activation messages. A consensus-side Pulse over a Nebula bundle root cannot be replayed as a Warp-envelope Pulse over `LUX-WARP-ZAP-PULSE-v1 || D`, and vice-versa.

Re-using one prefix across surfaces would let a consensus-side Pulse forge a cross-chain envelope (or one lane forge another); the distinct-tag discipline explicitly rejects this.

3.6 Why HashSuiteID is bound

Every Pulsar deployment pins a hash suite identifier (`Pulsar-SHA3` as the production default, `Pulsar-BLAKE3` as an optional non-normative profile). `HashSuiteID` is a field of `Message`, so it is part of `c14n(Message)` and therefore committed in D and authenticated by all lanes. Cross-suite signatures do not collide; a forger cannot leverage a signature under one suite to forge under another ([proofs/pulsar/hash-suite-separation.tex](#) Theorem ??).

At verification, the destination’s resolver returns the suite the source chain was using at signing time. If that suite does not match the envelope’s resolved `HashSuiteID (Message.HashSuiteOrDefault())`,

the verifier returns `ErrSuiteMismatch` ([15] `warp/pulsar/pulsar.go`). This catches the case where the source chain has Reanchored to a new suite but the destination has not yet seen the Reanchor: rather than verifying under a wrong suite, the destination correctly refuses.

3.7 One parser, no dispatcher

`ParseEnvelope` is the single envelope parser (`envelope.go`). There is no `UnsignedMessage/Message` split, no parallel envelope-version dispatcher, and no cross-version fallback. It reads, in order: the 5-byte magic (`expectMagic`); the kind byte, which must equal `kindEnvelope` (0x02); the `Message` (whose own leading byte must equal `zapKindMessage`, 0x01); the Beam (rejecting a non-canonical bitset); the `PulseSig` and `MLDSACertSet` length-prefixed frames; and finally asserts the cursor is exactly at end-of-buffer (no trailing bytes) before running structural `Verify`. `ParseMessage` is the analogous single parser for a standalone `Message`.

3.8 Backward compatibility

A legacy Warp 1.x RLP decoder presented with ZAP wire bytes rejects them: the stream begins with 'L' (0x4c), which is below RLP's list-prefix floor (0xc0–0xff), so it is not a valid RLP list. Conversely, `ParseEnvelope` presented with legacy RLP bytes (lead 0xc0–0xff) or the bare legacy 0x02 envelope byte rejects them at the magic check (`errZapBadMagic`). The rejection is mutual and total: the two wires never silently interoperate. This is a hard refusal, not a downgrade (regression-tested by `TestParseEnvelopeRejectsLegacyRLP`).

A sender that must reach legacy v1-only verifiers emits legacy v1 bytes on the legacy channel; the ZAP channel is independent. The ZAP Beam signs `LUX-WARP-ZAP-BEAM-v1 || D` over the full `Message` (lineage included), so it is *not* bit-equal to a legacy v1 BLS aggregate over the old unsigned body — the two channels are cryptographically as well as syntactically distinct.

3.9 Locally-originated messages

A message with no Pulsar lineage is not a separate envelope version: it is a `Message` with zero lineage fields (`SourceNebulaRoot` all-zero, `SourceKeyEraID` and `SourceGeneration` zero, `HashSuiteID` resolved to the default by `NewMessage`) and the two PQ lanes carried as empty `u32(0)` frames. The verifier observes the empty PQ lanes and applies the Beam-only path (`VerifyEnvelope`); the destination's downstream consumers choose whether to act on Beam-only or to require Pulse-backed (`RequirePulse` in `VerifyOptions`).

3.10 No parallel envelope version

The wire carries a format-version byte *inside* the magic (0x01) and a kind discriminator (0x02 = envelope). There is no separate envelope-version dispatch and no 0x03 reserved. Future extensions (Warp Private, Warp Private PQ) reuse the ZAP framing with additional length-prefixed fields, gated by the chain's governance — not by a new wire version. This rejects the “every protocol bump is a wire bump” anti-pattern: the wire is stable, the policy on top of it evolves.

3.11 Forward-only migration discipline

There is no v1 ↔ v2 negotiation, no protocol downgrade, and no shared-protocol fallback that lets a ZAP sender be “polite” to a legacy-only receiver: senders that need to reach legacy receivers emit legacy bytes on the legacy channel. The two wires coexist; the bytes do not negotiate, and they reject each other cleanly at the first byte. This is the LP-105 [16] deprecation timeline made wire-explicit: “no backwards compatibility only forwards perfection.” The format-version byte in the magic is for honest version identification, not for protocol-level fallback.

4 Cross-Chain Binding

A Warp 2.0 envelope’s signed `Message` folds three lineage scalars from the source chain (alongside `HashSuiteID`, Section 3). This section specifies what each one binds and how the destination verifier uses it. Because the scalars are fields of `Message`, they are committed in the digest $D = \text{keccak256}(\text{“LUX-WARP-ZAP-CORE-v1”} \parallel \text{c14n}(\text{Message}))$ and authenticated by every lane.

4.1 The three lineage scalars

- `sourceNebulaRoot`: the source chain’s canonical bundle root for the bundle that the Pulse signs. A 32-byte digest binding the source chain’s validator-set hash, parent epoch, and bundled vertices (Section ?? of [7]; equivalently `proofs/definitions/transcript-binding.tex` Definition ??).
- `sourceKeyEraID`: the source chain’s Pulsar group-key lineage ID at the time of signing. Bumps only at Reanchor. Stable across LSS Reshare / Refresh within the era.
- `sourceGeneration`: the source chain’s LSS resharding generation ID at the time of signing. Bumps every Refresh / Reshare under the same `GroupKey`.

The destination chain resolves (`sourceChainID`, `sourceKeyEraID`, `sourceGeneration`) to the source’s `GroupKey + HashSuiteID` via a `GroupKeyResolver` ([15] `warp/pulsar/pulsar.go`).

4.2 The GroupKey resolver

```
type GroupKeyResolver interface {
    ResolveGroupKey(
        sourceChainID [32]byte,
        keyEraID uint64,
        generation uint64,
    ) (gk *corona.GroupKey, suiteID string, err error)
}
```

The resolver is implemented by the destination chain against its source-chain key registry — a contract that records the source’s `GroupKey` lineage as it evolves through Bootstrap, Reshare, and Reanchor events. Returning a zero-pointer `GroupKey` or empty `suiteID` is treated as `ErrGroupKeyResolverFailed`.

Within an era. For every g within the same η , the `GroupKey` is byte-identical (Lemma ?? of `proofs/lss/lifecycle-safety.tex`). Therefore the resolver only needs to learn the source’s η on Reanchor; intermediate Refresh / Reshare events do not require new resolver state.

Across eras. A Reanchor event on the source chain produces a new η and a new `GroupKey`. The destination’s resolver must observe the Reanchor (typically by validating a destination-side update transaction that carries the Reanchor’s activation cert) before it can verify v2 envelopes signed under the new η .

Why this is not a centralized bridge oracle. The resolver does not introduce trust beyond the destination chain itself: the resolver’s state is on-chain on the destination, updated via destination-side transactions whose finality is itself Horizon-final on the destination. The trust model is destination Horizon + source Horizon (the two compose); there is no third-party oracle.

4.3 Transitive L1 / L2 / L3 inheritance

The L1 / L2 / L3 tier model (Section ?? of [7], Definition ??) extends to Warp 2.0 cross-chain envelopes through transcript binding:

Source = L1 (sovereign). The source chain’s Horizon-final binds its own validator-set Pulse + ML-DSA. The Warp 2.0 envelope’s Pulse inherits from the source’s **GroupKey** at (η, g) . The destination’s verifier returns Horizon-final acceptance iff Prism accepts.

Source = L2 (co-validating with primary N_P). The source chain’s activation transcript binds `primary_nebula_root` from N_P . The Warp 2.0 envelope’s Pulse over the source’s Nebula-Root transitively inherits from N_P ’s validator set: a destination receiving an L2 Warp message verifies the L2’s Pulse and, transitively, the L2’s anchor to its primary network — without trusting the L2 in isolation. The transitive inheritance is Theorem ?? of `proofs/quasar/11-12-covalidation.tex`, applied at the cross-chain layer.

Source = L3 (rollup / validity / app-execution). The source’s Horizon-final inherits from the underlying L1/L2’s Pulse + ML-DSA via the L3’s chosen proof system (Groth16, STARK, FHE circuit). The Warp 2.0 envelope from an L3 carries the L3’s Pulse over the L3’s bundle root; the destination’s resolver maps the L3’s `sourceKeyErald` to the underlying L1/L2’s **GroupKey** at the bound moment. The L3’s proof system is a compatibility / compression / privacy adapter, not the PQ root of trust (Remark 1, Section 5).

4.4 Cross-chain validator-set evolution

A common operational worry: “what if the source chain rotates its validator set between the time the Warp envelope is signed and the time it lands on the destination?” Warp 2.0’s answer: bind the (η, g) at signing, resolve the **GroupKey** accordingly. Validator rotation within an era does not change the **GroupKey** (Lemma ??), so the destination resolver can verify against the stable **GroupKey** $_\eta$ without reconciling the precise g . Across-era rotation (Reanchor) does change the **GroupKey**, and the destination’s resolver must observe the Reanchor before envelopes signed under the new η verify.

The discipline is symmetric: a source-chain Reanchor produces a new η ; envelopes signed under the new η are unverifiable on the destination until the destination’s resolver state has been updated; updating the resolver state is a destination-side transaction whose own finality is Horizon-final. There is no out-of-band bridge component.

4.5 Cross-chain Horizon batching

Multiple source messages may be packed into a single Pulse by signing over a Merkle root of their per-message digests. The batched Pulse signs “LUX-WARP-ZAP-PULSE-v1” || Merkle($D_1, D_2, \dots, D_k$), where each $D_i = \text{keccak256}(\text{“LUX-WARP-ZAP-CORE-v1”} \parallel \text{c14n}(\text{Message}_i))$ is an individual envelope’s digest:

```
BatchedPulseSubject := "LUX-WARP-ZAP-PULSE-v1"
                    || MerkleRoot(D_1, D_2, ..., D_k)
```

The destination provides Merkle proofs to verify individual message digests against the batched root. This is an extension; the shipped v1.22.0 wire carries a single **Message** per envelope.

4.6 Replay protection

A Warp 2.0 envelope is replay-safe across chain boundaries: the `sourceChainID` binds the originator, and the destination’s chain id is bound via the destination’s payload-level conventions (e.g., `AddressedCall` carries a destination chain id). At the source-finality level, the `sourceNebulaRoot`

pins the bundle: replaying the same envelope at a later source-bundle would not match the destination’s resolver because the resolver verifies the envelope’s (`sourceKeyEraID`, `sourceGeneration`) matches what the source had at the bundle.

The PQ lanes are not vulnerable to replay across protocols: the Warp envelope’s Pulse tag `LUX-WARP-ZAP-PULSE-v1` differs from the consensus-side tag `QUASAR-PULSAR-BUNDLE-v1`, so a Pulse over a bundle root cannot be lifted into an envelope (Section 3).

5 Soundness

The cross-chain soundness theorem is `proofs/quasar/warp-pq-soundness.tex`. We restate it and connect it to the per-lane reductions and to the consensus-side Horizon theorem.

5.1 Warp 2.0 envelope unforgeability

Theorem 1 (Warp 2.0 envelope unforgeability). *Let env_2 be a Warp 2.0 envelope (Definition ??) with successful verification under `VerifyWithOptions`. Then forging an alternative env'_2 on the same (`sourceChainID`, `sourceNebulaRoot`, `sourceKeyEraID`, `sourceGeneration`) tuple with a different payload requires breaking either:*

1. *BLS12-381 co-CDH (forging the Beam component); OR*
2. *Module-LWE / Module-SIS (forging the Pulsar Pulse, Theorem ??).*

A quantum adversary breaking only co-CDH (and not MLWE/MSIS) succeeds against the Beam alone; the Pulse remains unforgeable, so the v2 envelope as a whole remains sound under `VerifyWithOptions`.

The full proof is Theorem 1 of `proofs/quasar/warp-pq-soundness.tex`; the per-lane reductions live in `proofs/pulsar/unforgeability.tex` (Pulsar SUF-CMA) and are companion-cited from `quasar-cert-soundness.tex` [8]. We highlight three structural facts:

The conjunction is essential. The shape “`VerifyWithOptions` accepts iff Beam-OR-Pulse” is wrong for the same reason that “Horizon iff Beam-OR-Pulse” is wrong (Section ?? of [7]; [5]): under a quantum adversary that breaks only BLS, “Beam-OR-Pulse” degrades to “Beam alone” and the envelope becomes forgeable. Warp 2.0’s `VerifyWithOptions` is conjunctive: every required lane must verify against the same canonical digest D (each lane over its own $\text{DST} \parallel D$).

PQ finality lives in the Pulse. The Pulse is the post-quantum root of trust for a Warp 2.0 envelope. The Beam provides classical fast-path verification (sub-millisecond on commodity hardware). The optional ML-DSA cert set provides per-validator PQ accountability and slashing-evidence material. Under a quantum break of BLS, the Beam falls; the envelope remains sound iff the Pulse verifies. Without the Pulse, the v2 envelope degrades to Warp 1.x semantics — which is correct, because absent the Pulse, the envelope is exactly Warp 1.x.

The Groth16 wrapper is not part of the PQ reduction.

Remark 1 (Groth16 wrapper does not contribute PQ security to Warp 2.0). *A Z-Chain Groth16 rollup proof of ML-DSA cert-set verification (Definition ??) is a classical succinct proof of post-quantum signature verification. The proof system is pairing-based and not post-quantum. Warp 2.0’s PQ soundness (Theorem 1 parts 2 and the `Pulsar.Verify` branch) flows from the ML-DSA signatures and Pulsar threshold signature alone; the Groth16 proof is verified for compatibility / compression only and is not part of the PQ reduction. See Remark ?? of `proofs/quasar/horizon-soundness.tex`.*

The Mar-3 freeze ships Warp 2.0 with the Pulse as the PQ root of trust. A future Warp Private PQ envelope replaces Groth16 with P3Q [3] or a lattice-based wrapper; PQ finality continues to flow from ML-DSA + Pulsar.

5.2 Per-lane reductions

Beam (BLS aggregate). Forging the signed Message (the Beam carrier inside the envelope) requires either $> 2/3$ -weight BLS-key control, or a forgery in the BLS12-381 signature scheme. Reduces to co-CDH [1]. Quantum-broken in polynomial time.

Pulse (Pulsar SUF-CMA). Forging a Pulse over $\text{PulseSigningBytes}(D) = \text{“LUX-WARP-ZAP-PULSE-v1”}$ with at most $t-1$ corrupted parties reduces to MLWE / MSIS, by Theorem ?? of proofs/pulsar/unforgeability. With LP-073 parameters [6], classical security $\geq 2^{142}$ and quantum security $\geq 2^{130}$.

ML-DSA cert set (FIPS 204 EUF-CMA). Each per-validator signature reduces to MLWE / MSIS by FIPS 204’s standard analysis. The cert-set’s $\geq 2/3$ -weight requirement adds no structural shortcut beyond the per-validator forgery cost.

5.3 Cross-lane independence

Lemma 2 (Lane independence under quantum advice, cross-chain). *Let $\mathcal{A}$ be a polynomial-time adversary with quantum oracle access to BLS12-381’s discrete log. Then $\mathcal{A}$ ’s success probability against the Pulsar lane in a Warp 2.0 envelope is bounded by $\text{Adv}_{q, \sigma_E}^{\text{MLWE}}(\mathcal{A}) + \text{Adv}_{q, [\mathbf{A}] - c\tilde{\mathbf{b}}, 2B}^{\text{MSIS}}(\mathcal{A}) + \text{negl}(\lambda)$.*

This is the cross-chain analog of Lemma ?? of [7]: BLS provides no PQ contribution to a multi-lane cert, and the Pulse / ML-DSA reductions are independent of the BLS reduction.

5.4 Warp Private orthogonality

Remark 2 (Warp Private orthogonality). *Warp Private (FHE-encrypted payload, optionally Pulse-signed) is orthogonal to Warp 2.0. Warp Private provides confidentiality; Warp 2.0 provides PQ finality. They compose: a Warp Private envelope MAY carry a Pulse, in which case Theorem 1 applies unchanged to the encrypted-payload binding. Until the FHE circuits are production-ready, the Mar-3 freeze ships only Warp 2.0; Warp Private is open work.*

5.5 Mechanization status

- **Warp 2.0 envelope unforgeability.** proofs/quasar/warp-pq-soundness.tex.
- **Pulsar SUF-CMA.** Lean: Crypto.Pulsar.corona_pq_unforgeability.
- **Reshare preserves master secret.** Lean: Crypto.Threshold.Reshare.reshare_preserves_public_key.
- **LSS lifecycle invariants.** proofs/lss/lifecycle-safety.tex, proofs/lss/rollback-safety.tex.
- **Tamarin.** tamarin/QuasarConsensus.spthy verifies the protocol-level handshake / vote / bundle flow that produces the source-side Horizon certificate that Warp 2.0 carries.

5.6 Why we do not assume synchrony for Warp delivery

Warp envelope delivery is asynchronous; the destination’s verifier accepts an envelope when all required lanes verify against the same transcript, irrespective of inter-chain timing. There is no global clock assumption. A Warp 2.0 envelope sits in the destination’s pending-envelopes queue until the destination’s resolver has the source’s (η, g) resolved; once resolved, the envelope is verifiable.

This is the cross-chain analog of asynchronous PQ finality (Section ?? of [7]). The forward consequence: cross-chain finality lag is bounded by source-chain Horizon-final lag plus destination-chain resolver-update lag, both of which are governance-configurable.

6 Fuzz-Discovered Vulnerability: lattigo ReadUint64Slice DoS

The Warp 2.0 implementation went through a constant-time review and a fuzz harness pass before the Mar-3 freeze. The fuzz harness (`warp/pulsar/fuzz_pulse_test.go`, `warp/pulsar/fuzz_horizon_certificate_test.go`, `warp/fuzz_envelope_test.go`) surfaced one production-relevant vulnerability in the `luxfi/lattice/v7` [9] deserializer that the Pulsar wire format depends on. This section documents the finding, the exploit shape, and the fix.

6.1 The bug

The lattigo v7 fork carries `utils/buffer/reader.go` `ReadUint64Slice`, which recursively reads a uint64 length-prefix followed by a uint64 slice of the declared length. The recursion happens via a partial-peek buffer call: the reader peeks at the length, allocates a slice, and recurses to fill it. If the declared length is unbounded by the available bytes, the recursion descends without making progress.

The exploit. An attacker constructs a wire-format Poly with an inner length-prefix of `0xFFFFFFFFFFFFFFFF` (or any large value) and only a few bytes of payload. The lattigo deserializer reads the length, allocates a slice of the declared size, and recurses to fill it. The recursion descends without resolving; the Go runtime stack overflows; the process dies.

Where it lands. On the destination chain. Specifically: a destination receiving a Warp 2.0 envelope with an attacker-crafted Pulse byte stream, calling `warp/pulsar.DeserializePulse`, which delegates to the lattigo deserializer. The destination chain's verifier process panics; the chain's verifier loop crashes. Repeated envelopes denial-of-service the destination's Warp message ingress.

Severity. High. A non-validator attacker can DoS arbitrary destination chains by sending malformed Warp 2.0 envelopes. The Beam aggregate inside the envelope is irrelevant to the attack; the lattigo deserializer is invoked before the envelope is structurally valid. The attack surface is the public Warp 2.0 message channel.

6.2 The fuzz harness that found it

The fuzz harness at `warp/pulsar/fuzz_pulse_test.go` ([15], file head):

```
// FuzzPulseDeserialize runs DeserializePulse over arbitrary bytes and
// confirms it never panics.
func FuzzPulseDeserialize(f *testing.F) {
    // Seed: a real pulse from a 3-of-2 ceremony.
    good := mustGoodPulseSeed()
    f.Add(good)
    f.Add([]byte{})
    f.Add([]byte{0x00})
    f.Add(bytes.Repeat([]byte{0xFF}, 1024))
    // ...
    f.Fuzz(func(t *testing.T, raw []byte) {
```



```

    _, err := DeserializePulse(raw)
    // Property: errors are fine; panics are not.
    // (The defer-recover in DeserializePulse turns lattigo
    // panics into clean errors.)
    _ = err
})
}

```

The harness ran for 60 s under `go test -fuzz=FuzzPulseDeserialize` and quickly produced a corpus entry of ~ 30 bytes that triggered the lattigo recursion. The diagnostic was a stack overflow in `lattigo/v7/utils/buffer/reader.go:ReadUint64Slice`; the stack trace showed unbounded recursion through the length-prefix code path.

6.3 The fix

The fix lives in `warp/pulsar/pulsar.go` ([15] L257–L327 in the canonical commit) in three layers:

Layer 1: outer wire-size cap.

```
const MaxPulseWireSize = 64 * 1024
```

A real Pulsar lattice threshold signature is ~ 33 KB (LP-073 [12] §“Wire Format”); the cap is 64 KB to leave headroom for ring-parameter changes. The cap doubles as a recursion bound: lattigo’s `ReadUint64Slice` recurses by length field, so bounding the byte stream bounds the recursion depth ($64\text{ KB} \div 8 = 8192$ frames worst case, well under Go’s default 8 MB goroutine stack at 64 B per frame).

Layer 2: per-lane frame caps.

```
const MaxPulseFrameSize = 32 * 1024
const MaxLatticeUintSliceLen = 4096
```

`MaxPulseFrameSize` bounds each individual lane (C, Z, Δ) inside a serialized pulse. `MaxLatticeUintSliceLen` bounds the largest uint64 slice inside any lattigo wire frame. A canonical Pulsar Poly has 256 coefficients per level; a Vector / Matrix has at most $M \cdot N = 8 \cdot 32 = 256$ polys; every inner length-prefix should fit comfortably under 4096. A frame whose declared inner length exceeds this is a structural red flag and is rejected before lattigo’s `ReadFrom` can recurse.

Layer 3: end-to-end frame walker. `validatePolyFrame` and `validateVectorPolyFrame` parse the Poly / Vector[Poly] wire format byte-by-byte before calling lattigo. Every length-prefix is checked against `MaxLatticeUintSliceLen`; every nested coefficient slice is checked against the available bytes. If the frame is malformed (too short, length-prefix exceeds bound, trailing bytes), the validator returns a clean error and the lattigo deserializer is never invoked.

Layer 4: defer-recover boundary. The `DeserializePulse` entry point installs a `defer recover()` handler that converts any lattigo panic that escapes the validators into a clean error ([15] `warp/pulsar/pulsar.go` L389–L398):

```

defer func() {
    if r := recover(); r != nil {
        sig = nil
        err = fmt.Errorf("warp pulsar: deserialize panic recovered: %v", r)
    }
}()

```


This is defense in depth: the frame walker should catch all malformed inputs, but if a future lattigo change introduces a new panic path, the recover boundary ensures the destination chain’s verifier process does not crash.

6.4 Why all four layers

The combination is belt-and-suspenders by design:

- Layer 1 (wire-size cap) is the cheapest check; rejects 99% of attack inputs in $O(1)$.
- Layer 2 (per-lane caps) catches inputs that fit under the wire cap but blow up an inner frame.
- Layer 3 (frame walker) catches inputs whose lengths add up but whose internal structure recurses unboundedly.
- Layer 4 (defer-recover) catches any panic that escapes the validators (defense against future lattigo changes).

The performance cost is negligible: the frame walker walks the wire bytes once, in $\mathcal{O}(n)$ where n is the byte count, and the lattigo deserializer would walk the same bytes anyway. The only added overhead is the explicit length-prefix bound checks.

6.5 Upstream reporting

The bug is in `luxfi/lattice` (Lux’s pinned fork of `tuneinsight/lattigo` v7). On 2026-03-03 the Lux team filed the disclosure upstream as `luxfi/lattice` issue #2 (“DoS: ReadUint64Slice unbounded recursion + missing length cap”) with the regression seed `testdata/fuzz/FuzzPulseDeserialize/c` from `github.com/luxfi/warp`. The fix landed in PR #3 (“security(buffer): bound ReadUintNSlice + iterative rewrite”), which:

- replaces the recursion in `ReadUint{16,32,64}Slice` with an $O(1)$ -stack iterative loop;
- returns a new sentinel `ErrZeroProgress` when a `Peek` decodes zero elements while the destination still has capacity (the exact zero-progress recursion identified in §6);
- adds `Read{Uint16,Uint32,Uint64}SliceBounded` length-prefix-aware wrappers gated by `MaxSliceLen()` (default 1 MiB, configurable via the `LUX_LATTICE_MAX_UINT64_SLICE_LEN` environment variable).

Production deployments use the hardened `warp/pulsar/pulsar.go` validator stack regardless of upstream lattigo behavior, so a downstream lattigo change cannot regress the fix; the upstream patch is defense-in-depth for non-warp callers (ringqp deserialization, ciphertext unmarshalling, multiparty key shares).

6.6 Test surface

The fuzz harness continues to run in CI on every commit. The seed corpus includes:

- A valid Pulse from a real 3-of-2 ceremony.
- Empty bytes.
- Single 0x00 byte.
- Repeated 0xFF bytes (1024 bytes).
- Truncated Pulse prefixes (1, 7, 15 bytes from a real Pulse).
- The original ~ 30 -byte exploit input.

`FuzzPulseDeserialize`, `FuzzPulseSerialize`, `FuzzHorizonCertificate`, and `FuzzEnvelope` run for 60 s in CI; all four return zero panics on the v1.22.0 tag. The 30-byte exploit is in the corpus as a regression test; it returns a clean error and does not panic.

6.7 Lessons

- **Cross-language deserializers are a structural risk.** The lattigo wire format originated in a research library; the destination chain runs untrusted bytes through it. Bound

everything; trust nothing.

- **Fuzz harnesses pay for themselves.** The cost of writing the harness was hours; the cost of a destination-chain DoS in production would be days of incident response plus reputation damage.
- **Domain separation is necessary but not sufficient.** The fuzz finding is unrelated to the cryptographic soundness of the Warp 2.0 envelope shape (Section 5); it is a wire-format DoS. Both surfaces need attention.

The Mar-3 freeze ships with all four layers active. Subsequent fuzz finds are tracked in the same harness; new corpus entries are added on every find.

7 Implementation

The shipping implementation lives in two Go packages: github.com/luxfi/warp (the root package) and github.com/luxfi/warp/pulsar (the Pulse path). The split exists so the root package does not import the Pulsar kernel directly; the dispatch surface is the small `PulseVerifier` interface, with the concrete kernel-driven verifier in the subpackage.

7.1 Repository layout

Path	Role
<code>warp/codec.go</code>	ZAP profile constants: the four DSTs, <code>keccak256</code> , the digest <code>D</code> , <code>BeamSigningBytes</code> / <code>PulseSigningBytes</code> / <code>MLDSASigningBytes</code> , and the size caps
<code>warp/zap.go</code>	the canonical TLV codec: <code>wireMagic</code> , the append-primitives, the strict <code>zapReader</code> , and the bitset-canonicity helpers
<code>warp/envelope.go</code>	<code>Envelope</code> , <code>NewEnvelope</code> , <code>ParseEnvelope</code> , <code>VerifyEnvelope</code> , <code>VerifyWithOptions</code> , <code>VerifyPQLanes</code> , and the wire-size guards
<code>warp/message.go</code>	<code>Message</code> (the single signed subject; folds the former unsigned body + PQ lineage), <code>NewMessage</code> , <code>ParseMessage</code> , <code>VerifyWeight</code>
<code>warp/signature.go</code>	<code>BitSetSignature</code> (the Beam lane), <code>SignMessage</code> , <code>AggregateSignatures</code>
<code>warp/validator.go</code>	validator-set / canonical validator-set machinery
<code>warp/fuzz_envelope_test.go</code>	<code>FuzzEnvelope</code> / <code>FuzzEnvelopeRaw</code> : the envelope-parser fuzz harnesses
<code>warp/pulsar/pulsar.go</code>	<code>KernelVerifier</code> (Pulse over <code>PulseSigningBytes(D)</code>), <code>SerializePulse</code> / <code>DeserializePulse</code> , frame walkers, hardened deserializer
<code>warp/pulsar/classification.go</code>	<code>HorizonCertificate</code> helper (lifts envelope + transcript fields into the LP-105 cert shape)
<code>warp/pulsar/fuzz_pulse_test.go</code>	<code>FuzzPulseDeserialize</code> , <code>FuzzPulseSerialize</code> : the Pulse wire-format fuzz harness
<code>warp/pulsar/fuzz_horizon_certificate_test.go</code>	<code>FuzzHorizonCertificate</code> : the Horizon cert fuzz harness
<code>warp/pulsar/groth16_classification_test.go</code>	<code>TestIsClassified</code> : the optional Groth16 wrapper is correctly classified as classical (<code>IsPQRootOfTrust</code> returns false)
<code>warp/pulsar/hashsuite_mismatch_test.go</code>	<code>TestIsSuiteMismatch</code> when the resolver-supplied suite differs from the envelope's <code>HashSuiteID</code>

7.2 The single parser

The root `warp` package exposes one envelope parser, `ParseEnvelope`. There is no version-byte dispatcher and no cross-version fallback: `ParseEnvelope` requires the ZAP magic and the `kindEnvelope` byte, then reads the fields in order through a strict, bounds-checked cursor (`zapReader`).

```
func ParseEnvelope(b []byte) (*Envelope, error) {
    if len(b) == 0 { return nil, ErrEnvelopeEmpty }
    if len(b) > MaxEnvelopeSize { return nil, ErrEnvelopeTooLarge }

    r := newZapReader(b)
    if err := r.expectMagic(); err != nil {          // "LWZP"||0x01
        return nil, err
    }
    kind, err := r.u8()
    if err != nil { return nil, err }
    if kind != kindEnvelope {                       // 0x02
        return nil, fmt.Errorf("%w: envelope kind 0x%02x",
            ErrInvalidMessage, kind)
    }

    e := &Envelope{}
    if e.Message, err = parseMessage(r); err != nil { return nil, err }
    if e.Beam, err = parseBeam(r); err != nil { return nil, err }
    if e.PulseSig, err = r.varbytes(); err != nil { return nil, err }
    if e.MLDSACertSet, err = r.varbytes(); err != nil { return nil, err }
    if err := r.end(); err != nil { return nil, err } // reject trailing
    if err := e.Verify(); err != nil { return nil, err }
    return e, nil
}
```

Legacy RLP bytes (lead `0xc0–0xff` or the bare `0x02`) fail `expectMagic` and are rejected; there is no in-band lift of legacy v1 bytes into the envelope. `ParseMessage` is the analogous single parser for a standalone `Message`.

7.3 The PulseVerifier interface

```
type PulseVerifier interface {
    VerifyPulse(env *Envelope) error
}
```

The root package's `VerifyWithOptions` (and the PQ-only entrypoint `VerifyPQLanes`) accept a `PulseVerifier` and an `MLDSACertSetVerifier` through a `VerifyOptions` struct. Production code wires `warp/pulsar/KernelVerifier` (the concrete Pulsar-kernel-driven verifier). Tests wire a stub `PulseVerifier` that records calls without invoking the kernel.

The split keeps the import graph clean: the root `warp` package does not import the Pulsar kernel (github.com/luxfi/corona), so it has no dependency on the lattigo deserializer; downstream consumers that want only the Beam path do not pay for the lattigo dependency. The concrete `KernelVerifier` lives in the `warp/pulsar` subpackage, which imports the kernel.

7.4 The KernelVerifier flow

1. The envelope must carry a non-empty Pulse (`env.HasPulse()`); otherwise `ErrPulseAbsent`. (The Beam-only path never reaches here — `VerifyPQLanes` skips an absent lane unless `RequirePulse`.)
2. Resolve (`SourceChainID`, `SourceKeyEraID`, `SourceGeneration`) via the `GroupKeyResolver` to obtain (`GroupKey`, `suiteID`).
3. Check `suiteID` matches the envelope’s resolved `HashSuiteOrDefault()`; return `ErrSuiteMismatch` on mismatch.
4. Recompute $D = \text{env.Message.ID}()$ and build the signing bytes `PulseSigningBytes(D) = “LUX-WARP-ZAP-PULSE-v1” || D`. Because D folds `SourceNebulaRoot` / `SourceKeyEraID` / `SourceGeneration` / `HashSuiteID` / `SourceChainID` / `NetworkID` / `Payload`, verifying over these bytes binds the Pulse to every one of them.
5. Validate the Pulse wire bytes via the four-layer guards (Section 6): outer wire-size cap, per-lane frame cap, frame walker, defer-recover.
6. Deserialize via `DeserializePulse` into a `corona.Signature`.
7. Verify via `corona.Verify(GroupKey, PulseSigningBytes(D), signature)`; return `ErrPulseVerifyFailed` on rejection.

The Beam verification (BLS aggregate over “LUX-WARP-ZAP-BEAM-v1” || D) is run by `VerifyEnvelope`; the Pulse and ML-DSA lanes are additive. A destination chain that wants the full Horizon path wires all lanes via `VerifyWithOptions` and Prism-binds them via the `HorizonCertificate` helper (`warp/pulsar`).

7.5 Test status

The v1.22.0 tag on github.com/luxfi/warp carries the following test status:

- `warp/envelope_test.go`: 100% pass. Round-trip; empty optional lanes; legacy-RLP rejection; trailing-byte rejection; non-canonical-bitset rejection; `VerifyWithOptions` / `VerifyPQLanes` lane coverage.
- `warp/message_test.go`: 100% pass. `TestMessageIDIsLegacyKeccak` (digest is legacy Keccak, not SHA3); `TestLaneSigningBytesDistinct` (the four DSTs separate); per-field ID sensitivity; bad-kind / trailing-byte rejection.
- `warp/envelope_negative_test.go`: 100% pass. Message-transcript mutation rejection; explicit-vs-empty `HashSuiteID` binds distinctly.
- `warp/zap_test.go` / `warp/fuzz_envelope_test.go` / `warp/fuzz_signature_test.go`: zero panics in 60 s (`FuzzZapReader`, `FuzzEnvelope`, `FuzzEnvelopeRaw`, `FuzzBeamCodec`, `FuzzCorruptedMLDSACe`).
- `warp/pulsar/pulsar_test.go`: 100% pass. End-to-end ceremony + sign + serialize + deserialize + verify; tampered-envelope-field rejection.
- `warp/pulsar/fuzz_pulse_test.go`: zero panics in 60 s.
- `warp/pulsar/fuzz_horizon_cert_test.go`: zero panics in 60 s.
- `warp/pulsar/groth16_classification_test.go`: 100% pass. The optional Groth16 wrapper is classified as classical.
- `warp/pulsar/hashsuite_mismatch_test.go`: 100% pass. `ErrSuiteMismatch` returned on mismatch.

The build / test gate is `go test -count=1 ./...` from the `warp/` root.

7.6 Performance

Operation	Time
Beam-only envelope parse + verify (BLS aggregate, $n = 21$)	$\sim 875 \mu\text{s}$
Envelope parse (no Pulse)	$\sim 30 \mu\text{s}$
Envelope parse + Pulse deserialize + verify ($K = 21$)	$\sim 5 \text{ ms}$
Frame walker (<code>validatePolyFrame</code>) on 33 KB Pulse	$\sim 50 \mu\text{s}$
HashSuiteID resolver lookup (cached)	$\sim 1 \mu\text{s}$

The Pulse verify dominates full-envelope verification cost; the Beam aggregate verify is a few-percent overhead. Measurements on Apple M1 Max single core; full numbers and reproduction commands in the benchmark report [4].

7.7 Cross-implementation parity

The Go canonical ([15]) is the byte oracle. The wire format is normative; any other-language implementation must produce byte-equal envelopes. The KAT vectors gate every release: fuzz seeds + a representative real-ceremony Pulse + the Mar-3 exploit input, all checked into `warp/pulsar/testdata/`.

7.8 Wire-format / lane tests

- **TestParseEnvelopeRejectsLegacyRLP**: `ParseEnvelope` rejects legacy RLP bytes at the magic check (the wire boundary is mutual — legacy decoders likewise reject the ZAP magic because `'L' = 0x4c` is below RLP’s `0xc0` list floor).
- **TestEnvelopeRoundTrip**: `Envelope.Bytes()` $\rightarrow$ `ParseEnvelope` produces a byte-equal envelope.
- **TestEnvelopeEmptyOptionalLanes**: an envelope with both PQ lanes carried as empty `u32(0)` frames round-trips and verifies on the Beam-only path.
- **TestParseEnvelopeRejectsTrailingBytes / TestParseEnvelopeRejectsNonCanonicalBitset**: canonicity enforcement on decode.
- **TestMessageIDIsLegacyKeccak / TestLaneSigningBytesDistinct**: the digest D is legacy Keccak and the four lane DSTs are pairwise distinct.

All pass on the v1.22.0 tag.

8 Related Work

We compare Warp 2.0 to four cross-chain messaging protocols: Chainlink CCIP, Wormhole, IBC, and LayerZero. The structural lens is the cryptographic basis for source-chain finality: each protocol’s source-finality story has a different shape, and Warp 2.0’s contribution is making that story *post-quantum* via the Pulse, not classical-only via a BLS aggregate or oracle attestation.

8.1 Chainlink CCIP

CCIP [17] couples a Chainlink-operated oracle network with a separate Risk Management Network (RMN) for cross-chain message attestation. The cross-chain envelope is signed by the oracle network; the RMN provides a separately signed risk-management attestation.

Source-finality basis. Oracle attestation. The destination chain trusts that the oracle network correctly observed the source chain’s finality and signed accordingly. The trust model is the union of the oracle network’s classical signature scheme and the RMN’s classical signature scheme.

Differences vs Warp 2.0. (1) CCIP’s source finality is oracle-attested; Warp 2.0’s is source-validator-signed (the source’s own validator set produces the Pulse). (2) CCIP is classical-only; Warp 2.0 is hybrid PQ via the Pulse. (3) CCIP composes two oracle networks (Chainlink + RMN); Warp 2.0 composes three signing lanes (Beam + ML-DSA + Pulse) on the same source-validator set.

Where they could meet. A CCIP-style oracle-attested envelope could carry a Pulsar Pulse attesting the oracle’s view; the destination would verify the oracle’s PQ-signed view rather than its classical-signed view. The integration is straightforward; the open work is on the CCIP side, not Warp 2.0’s.

8.2 Wormhole

Wormhole [19] uses a 19-validator “guardian” network that signs cross-chain Verified Action Approvals (VAAs). The destination chain verifies the guardian quorum signature against the Wormhole guardian-key registry.

Source-finality basis. Guardian-network attestation. The destination trusts that the guardians correctly observed the source’s finality and signed.

Differences vs Warp 2.0. (1) Wormhole’s source finality is guardian-attested (a separate 19-of-13 committee); Warp 2.0’s is source-validator-signed (the source chain’s own validator set produces the Pulse, no separate committee). (2) Wormhole is classical-only (Ed25519 guardian keys); Warp 2.0 is hybrid PQ. (3) Wormhole’s guardian set is fixed-size and externally curated; Warp 2.0’s source-validator set is whatever the source chain runs, with LSS-managed dynamic resharing.

Wormhole 2.x and PQ. Wormhole has signaled intent to add PQ signatures to the guardian quorum. The shape would be analogous to Warp 1.x $\rightarrow$ 2.0 evolution. The structural difference remains: Wormhole’s source finality routes through a separate guardian set; Warp 2.0’s routes through the source’s own validators.

8.3 IBC

IBC [2] (Inter-Blockchain Communication) uses light clients on the destination chain that follow the source chain’s consensus headers. Source finality is verified by the destination running a verifier of the source’s consensus protocol.

Source-finality basis. Direct consensus verification. The destination’s IBC light client tracks the source’s consensus state (Tendermint’s prevote / precommit aggregate) and accepts cross-chain envelopes that the source has finalized.

Differences vs Warp 2.0. (1) IBC requires the destination to run a full source-consensus verifier; Warp 2.0 requires only a `GroupKeyResolver` keyed by source (η, g) . (2) IBC is classical-only (Tendermint Ed25519); Warp 2.0 is hybrid PQ. (3) IBC’s verification cost on the destination scales with the source’s validator set size and per-block signature verification; Warp 2.0’s verification cost is constant in the source’s validator set size (one Pulse verify, one BLS aggregate verify, one ML-DSA cert-set verify or one Groth16 verify).

The light-client tradeoff. IBC’s tightest binding to source finality is a strength: the destination can verify any source-finality property without trusting an oracle. The cost is operational: every source chain requires a separate IBC light client implementation on every destination. Warp 2.0 trades the tightest binding for a uniform **GroupKeyResolver** interface; the destination only needs to know the source’s GroupKey lineage, not its full consensus rules.

8.4 LayerZero

LayerZero [20] couples an oracle (delivering source block headers) with a relayer (delivering cross-chain message proofs against those headers). The destination chain accepts a message iff the oracle and relayer agree.

Source-finality basis. Oracle + relayer agreement. The destination trusts that the oracle correctly observed the source’s finality and that the relayer’s proof is valid against the oracle’s header.

Differences vs Warp 2.0. (1) LayerZero’s source finality routes through two separate trusted parties; Warp 2.0’s routes through the source’s own validator set. (2) LayerZero is classical-only; Warp 2.0 is hybrid PQ. (3) LayerZero’s trust model is two-of-two oracle / relayer; Warp 2.0’s is the source’s own $\geq 2/3$ -weight validator quorum.

8.5 Comparison table

Protocol	Source-finality basis	PQ?	Trust model
CCIP	Oracle network + RMN	no	Oracle network classical sigs $\cup$ RMN classical sigs
Wormhole	19-validator guardian network	no	Guardian quorum classical sigs
IBC	Direct source-consensus verifier (light client)	no	Source’s classical consensus correctness
LayerZero	Oracle (block header) + relayer (message proof)	no	Oracle classical sigs $\cap$ relayer correctness
Warp 1.x	Source validator BLS aggregate	no	Source’s $\geq 2/3$-weight BLS
Warp 2.0	Source validator Pulse + Beam + ML-DSA cert set	yes (hybrid)	Source’s $\geq 2/3$-weight Pulse + Beam + ML-DSA, conjunctive Prism predicate

The Warp 2.0 row is the only design that makes source-finality post-quantum and routes through the source’s own validators. The combination is the contribution.

8.6 Where Warp 2.0 sits in the cross-chain design space

Warp 2.0’s design space coordinates:

- **Trust routing:** source validators (Warp 1.x / 2.0) vs separate committee (Wormhole) vs oracle network (CCIP, LayerZero) vs direct light-client verification (IBC).
- **Cryptographic basis:** classical aggregate (BLS, Ed25519) vs hybrid PQ (Pulse + Beam + ML-DSA).
- **Source-finality binding:** NebulaRoot + KeyEra/Generation lineage (Warp 2.0) vs block header (IBC) vs payload-only (Warp 1.x, CCIP, Wormhole, LayerZero).

Warp 2.0’s coordinates are: source validators, hybrid PQ, NebulaRoot + lineage. The space is not exhaustively covered; the four neighbors illustrate the trade-offs but do not bound the

design space. A future protocol could combine, e.g., a source-validator routing with a STARK proof of finality (no `GroupKeyResolver` needed); the open work is the production deployment of such systems, and the conditions under which they outperform Warp 2.0.

8.7 Status

Warp 2.0 ships in the Mar-3 freeze. Warp 1.x continues to ship for backwards compatibility. The wire is forward-only; the two channels coexist without negotiation. Cross-chain destinations on Lux mainnet (Lux $\rightarrow$ Hanzo, Lux $\rightarrow$ Zoo, etc.) verify v2 envelopes against the source chain's Pulsar GroupKey lineage as recorded in their on-chain source-key-registry contracts. The PaaS deployment path [16] updates the registry contracts as the source chains' Reanchor lifecycles dictate; no out-of-band oracle is required.

References

- [1] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In *ASIACRYPT*. Springer, 2001.
- [2] Cosmos Network. Inter-blockchain communication protocol (IBC). <https://ibcprotocol.org>, 2021.
- [3] Lux Industries Inc. luxfi/p3q: Post-quantum plonky3 with sha3 challenger. <https://github.com/luxfi/p3q>, 2026.
- [4] Zach Kelling. Lux post-quantum consensus benchmarks. Lux Industries, Inc.; `papers/lux-pq-consensus-benchmarks/`, 2026.
- [5] Zach Kelling. Post-quantum finality for lux quasar. Lux Industries; `lux/proofs/pq-finality-no-bls.tex`, 2026.
- [6] Zach Kelling. Pulsar: Dynamic lattice threshold signatures for permissionless validator sets. Lux Industries, Inc.; `papers/lp-073-pulsar/`, 2026.
- [7] Zach Kelling. Quasar horizon: Hybrid post-quantum finality over dag-native nebula roots. Lux Industries, Inc.; `papers/lux-quasar-horizon/`, 2026.
- [8] Zach Kelling. QuasarCert soundness: Canonical formal specification, quasar 3.0. Lux Industries; `lux/proofs/quasar-cert-soundness.tex`, 2026.
- [9] Lux Core Team. Lattigo v7: A lattice-based cryptography library (lux fork). <https://github.com/luxfi/lattigo>, 2026. Fork of `tuneinsight/lattigo`; pinned at v7 for byte-exact reproducibility.
- [10] Lux Core Team. Lp-020: Quasar consensus 3.0. <https://github.com/luxfi/lps/blob/main/LP-020-quasar-consensus.md>, 2026.
- [11] Lux Core Team. Lp-021: Warp cross-chain messaging (1.x). <https://github.com/luxfi/lps/blob/main/LP-021-warp-messaging.md>, 2026.
- [12] Lux Core Team. Lp-073: Pulsar — dynamic lattice threshold signatures for permissionless validator sets. <https://github.com/luxfi/lps/blob/main/LP-073-corona.md>, 2026.
- [13] Lux Core Team. Lp-075: Bls aggregate signatures for warp messaging. <https://github.com/luxfi/lps/blob/main/LP-075-bls-aggregate.md>, 2026.

- [14] Lux Core Team. Lp-077: Linear shamir’s secret sharing — universal threshold lifecycle. <https://github.com/luxfi/lps/blob/main/LP-077-linear-shamir.md>, 2026.
- [15] Lux Core Team. Warp: Cross-chain messaging protocol (go canonical). <https://github.com/luxfi/warp>, 2026.
- [16] Lux Industries. LP-105: The lux finality stack — public vocabulary and internal names. Lux Improvement Proposal, 2026.
- [17] Sergey Nazarov et al. Chainlink cross-chain interoperability protocol (CCIP). In *Chainlink Whitepaper*, 2023.
- [18] Vishnu J. Seesahai and Zach Kelling. LSS: A universal lifecycle framework for threshold cryptographic protocols. Lux Industries, Inc., 2026.
- [19] Wormhole Foundation. Wormhole: Cross-chain messaging protocol. <https://wormhole.com>, 2022.
- [20] Ryan Zarick, Bryan Pellegrino, and Caleb Banister. Layerzero: An omnichain interoperability protocol. <https://layerzero.network/whitepaper>, 2022.